Chapter 6
Registers and Counters

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Chapter Objectives

- Understand the use, functionality, and modes of operation of registers, shift registers (移位暫存器), and universal shift registers.
- Know how to properly create the effect of a gated clock (閘控時脈).
- Understand the structure and functionality of a serial adder circuit.
- Understand the behavior of a (a) ripple counter, (b) synchronous counter, (c) ring counter, and (d) Johnson counter.
  漣波計數器 同步计數器 環形计數器 詹森 计數器
- Be able to write structural and behavioral HDL models of registers, shift registers, universal shift registers, and counters.

6.1 Registers (1/3)

- Clocked sequential circuits (CSC) (時控序向電路)
  - A group of flip-flops and combinational gates connected to form a feedback path.

    Flip-flops + Combinational gates

    正反器 (essential)　　組合邏輯 (optional)
- Based on the functions, CSCs are classified 利用功能的不同將CSC分類……
  - Register (暫存器)
    - Flip-flops hold the binary information.
    - Combinational ates determine how the information is transferred into the register.

      將資料在暫存器間轉移
  - Counter (計數器)
    - A special register that goes through a predetermined sequence of binary states.
    - The combinational gates are connected in such a way as to produce the prescribed sequence of states.

6.1 Registers (2/3)

＊D型 正反器→透通形正反器
輸入直接反应給輸出

- An *n*-bit register
  - *n* flip-flops capable of storing *n* bits of binary information
- Consider the 4-bit register in Figure 6.1,
  - If the contents of the register must be left unchanged,
    - The inputs must be held constant.
      - This would result in the data bus (匯流排) driving the register being unavailable for other traffic.
    - The clock must be inhibited from the circuit by inserting some control gates into the clock path.
      - This would result in uneven propagation delays between the global clock and the inputs of flip-flops.
  - Solution: Gated clock 閘控時脈(沒有真正控制Clock，而是利用D形正反器設計輸入來達成控制Clock的效果。)
    - Control the operation of the register with the *D* inputs, rather than the clock in the *C* inputs. 不能直接控制Clock，会造成時間差。

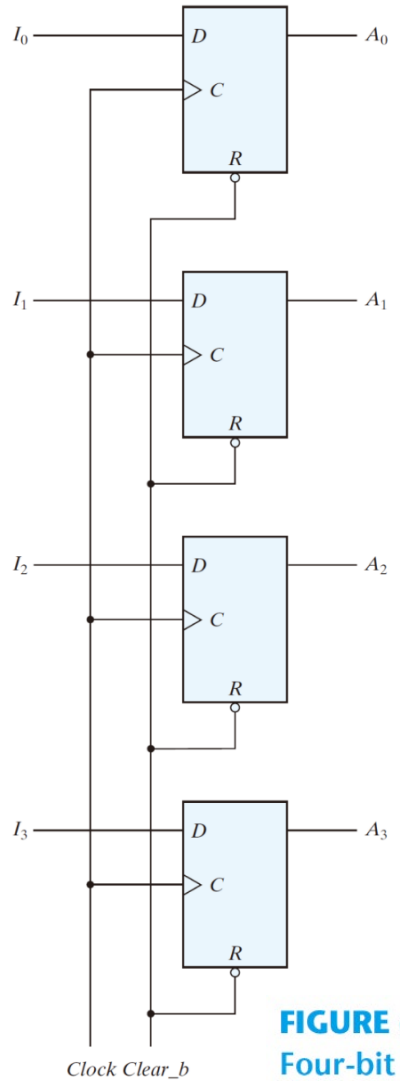


FIGURE 6.1
Four-bit register

6.1 Registers (3/3)

- 4-bit register with parallel load
  - The transfer of new information into a register is referred to as *loading* or *updating*.
  - If all the bits of the register are loaded simultaneously with a common clock pulse, the loading is regarded as *parallel*.

4位元並行載入暫存器

其实就是多工器!

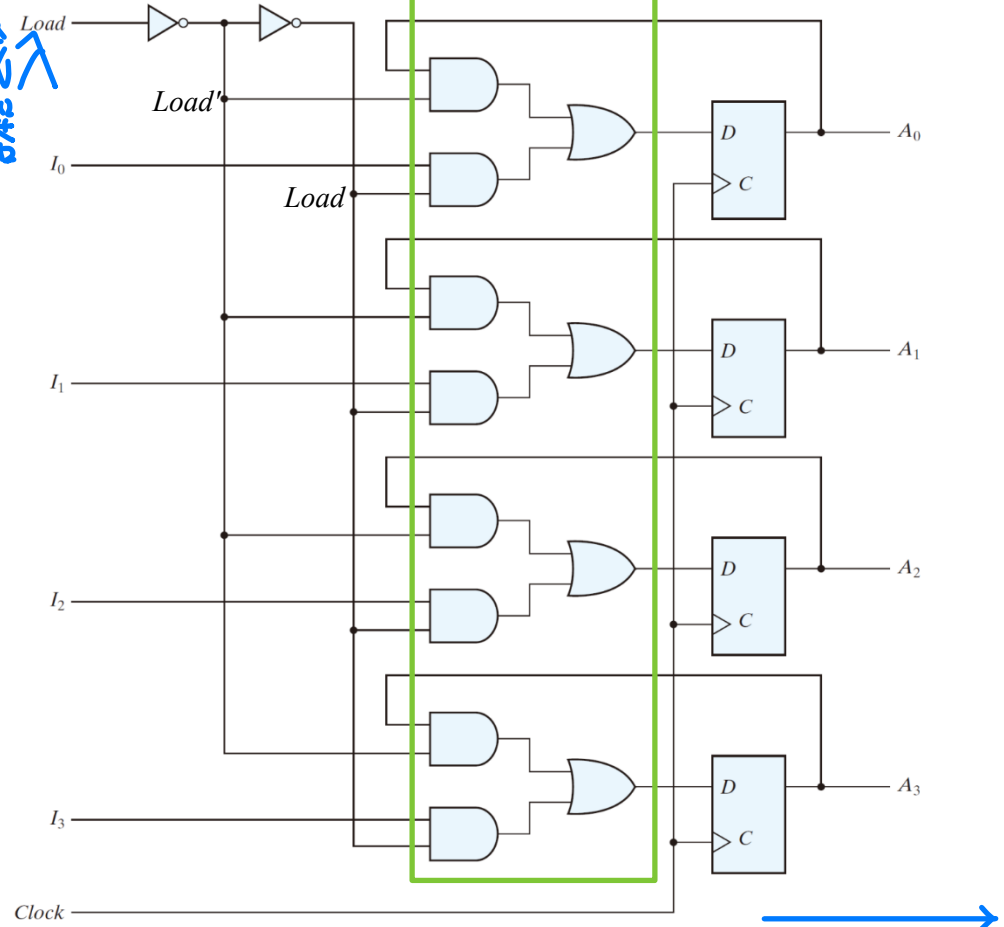
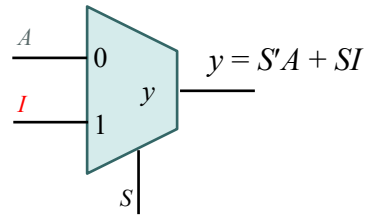
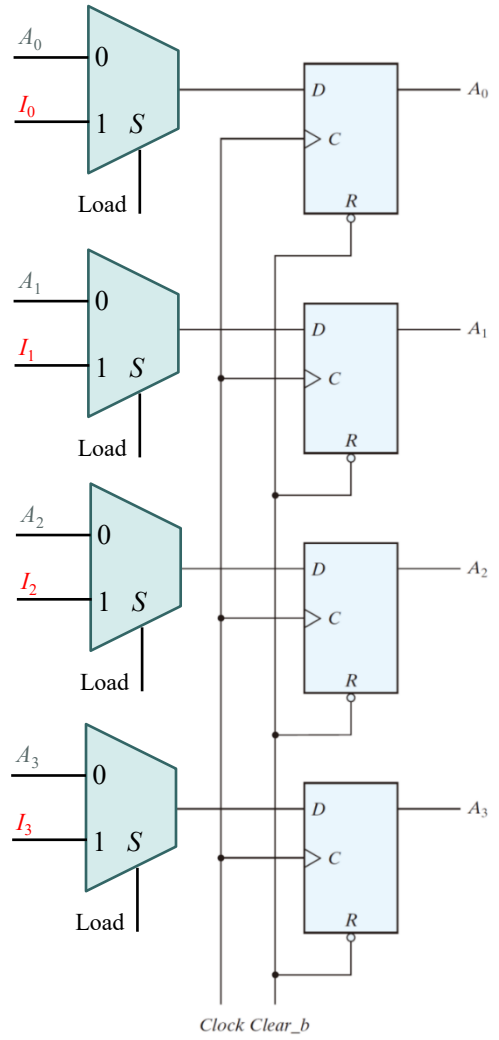


FIGURE 6.2
Four-bit register with parallel load





多工器

许多人(輸入源)共用一條線

↑

*匯流排 Bus

6.2 Shift Registers (1/12) 移位暫存器

- Shift register
  - a register capable of shifting its binary information in one or both directions
- Simplest shift register

資料会往右移动

資料会往左、往右移动

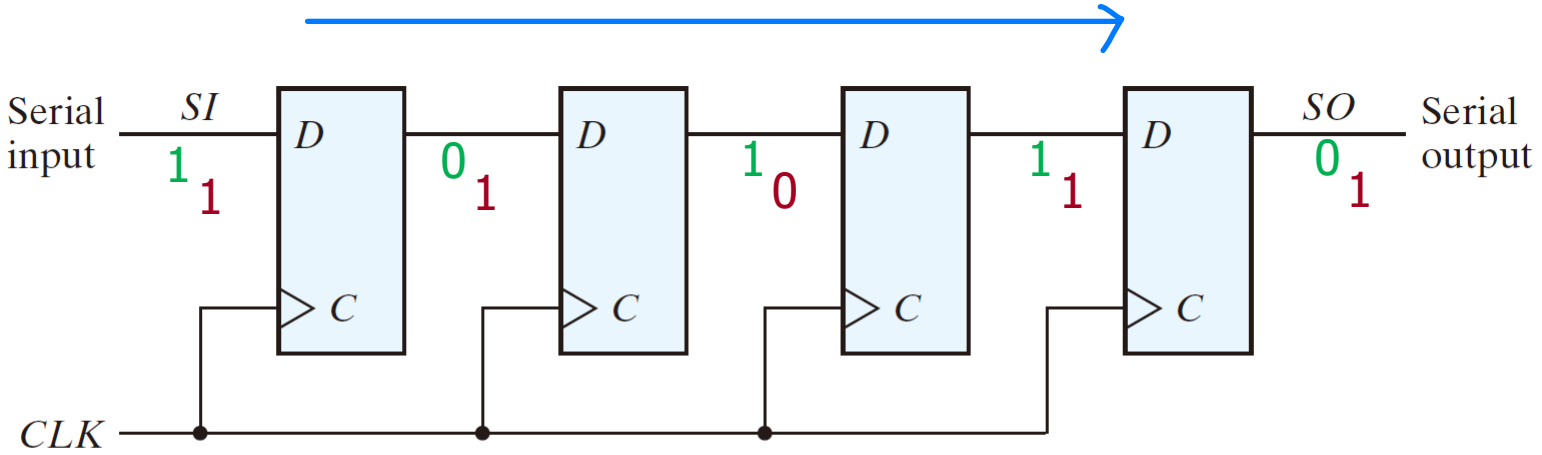


FIGURE 6.3
Four-bit shift register

6.2 Shift Registers (2/12)

- **Practice Exercise 6.1 Objective**: Draw the logic diagram of a circuit that suspends the clock action of a *D* flip-flop without gating its clock. Describe the behavior of the circuit.

  Answer:

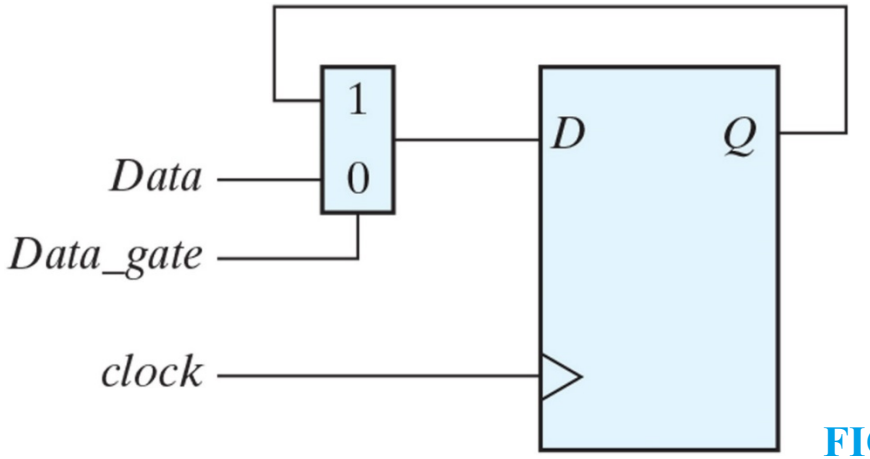


FIGURE PE6.1

If *Data_gate* is 0, *Data* is transferred to *Q* with each active edge of *clock*. If *Data_gate* is 1, the value of *Q* is recirculated through the mux-flip-flop path, with the effect that *clock* appears to be suspended.

6.2 Shift Registers (3/12)

- Serial transfer 串列傳輸
  - Information is transferred one bit at a time bt shifting the bits out of the source register into the destination register
- Parallel transfer
  - All the bits of the register are transferred at the same time

6.2 Shift Registers (4/12)
Serial Transfer

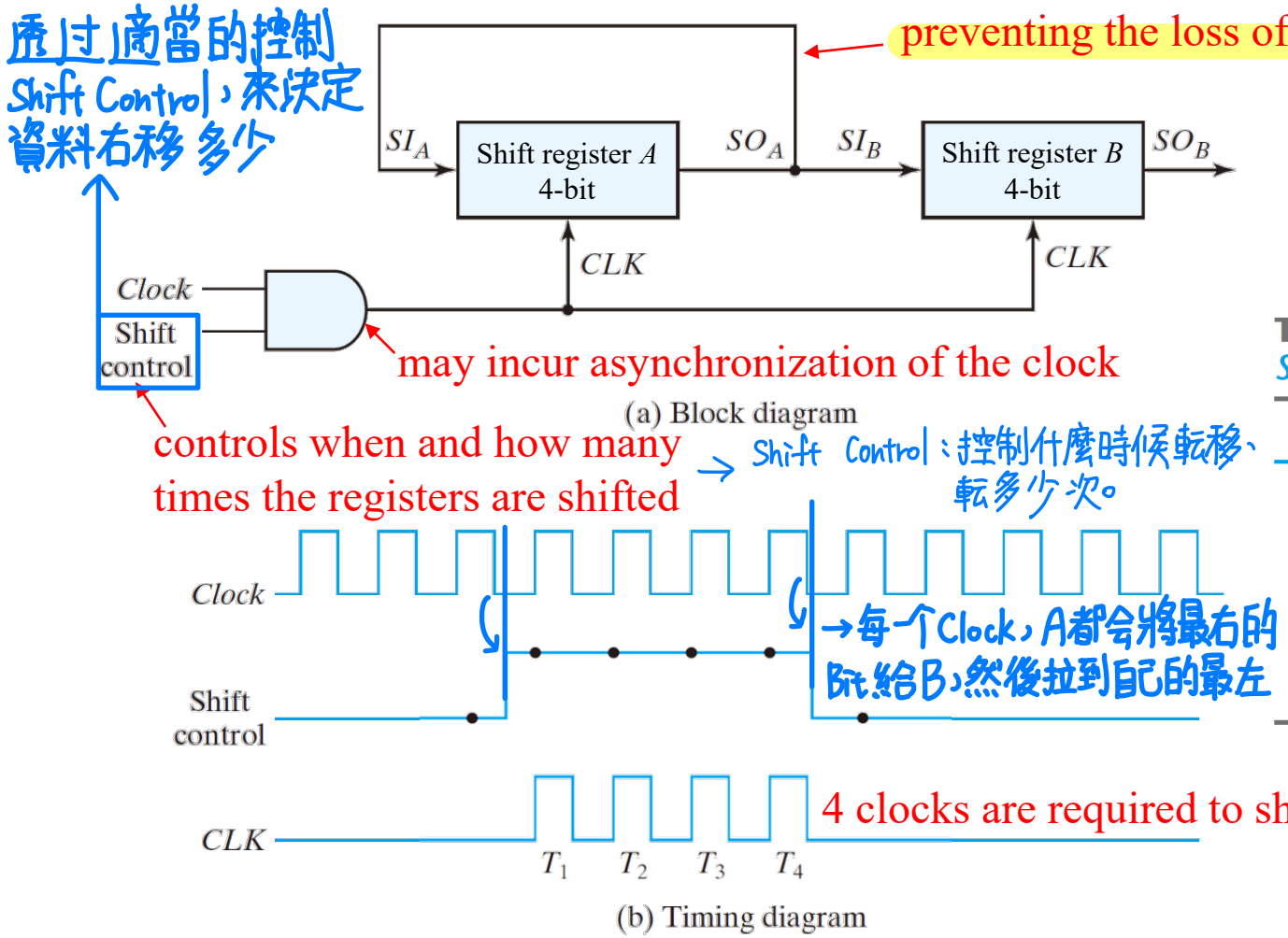


Table 6.1
Serial-Transfer Example

Timing Pulse	Shift Register A				Shift Register B			
Initial value	1	0	1	1	0	0	1	0
After T_1	1	1	0	1	1	0	0	1
After T_2	1	1	1	0	1	1	0	0
After T_3	0	1	1	1	0	1	1	0
After T_4	1	0	1	1	1	0	1	1

FIGURE 6.4
Serial transfer from register A to register B

6.2 Shift Registers (5/12)
Serial Addition

- Serial addition using *D* flip-flops
  - Assume $Q = 0$ initially.

$$A <= A + B$$

$$\begin{array}{r} \ 1\ \ 1\ \ 1 \\ \ 0\ 1\ 0\ 1 \\ +\ \ 0\ 0\ 1\ 1 \\ \hline \ 1\ 0\ 0\ 0 \end{array}$$

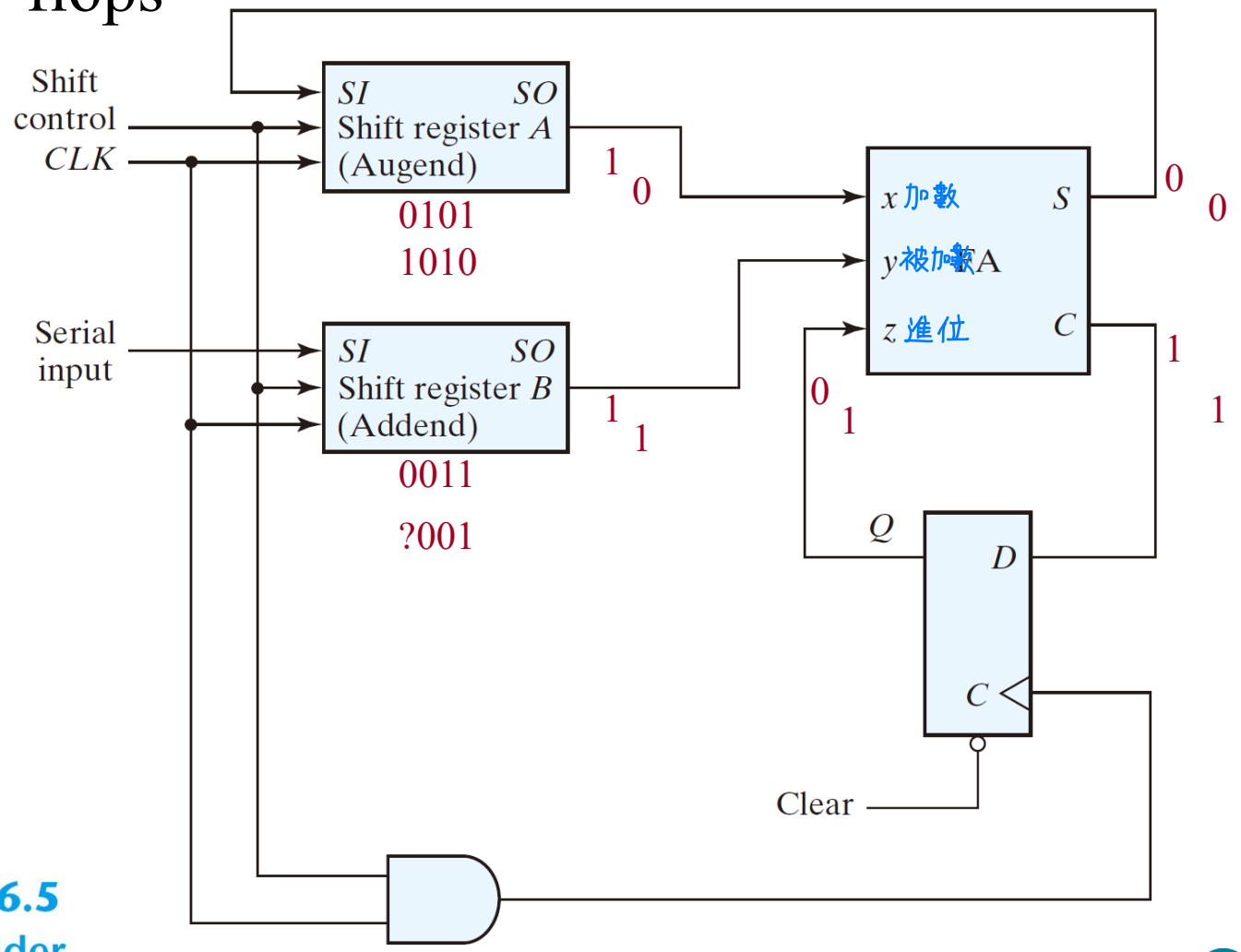


FIGURE 6.5
Serial adder

6.2 Shift Registers (6/12)
Serial Addition

- Serial adder using *JK* flip-flops

$$J_Q = xy$$
$$K_Q = x'y' = (x + y)'$$
$$S = x \oplus y \oplus Q$$

- Try to redesign with
  - *D* F/F
  - *T* F/F

Table 6.2
State Table for Serial Adder

Present State	Inputs		Next State	Output	Flip-Flop Inputs	
Q	x	y	Q	S	J_Q	K_Q
0	0	0	0	0	0	X
0	0	1	0	1	0	X
0	1	0	0	1	0	X
0	1	1	1	0	1	X
1	0	0	0	1	X	1
1	0	1	1	0	X	0
1	1	0	1	0	X	0
1	1	1	1	1	X	0

6.2 Shift Registers (7/12)
Serial Addition

- Circuit diagram

$$J_Q = xy$$
$$K_Q = x'y' = (x + y)'$$
$$S = x \oplus y \oplus Q$$

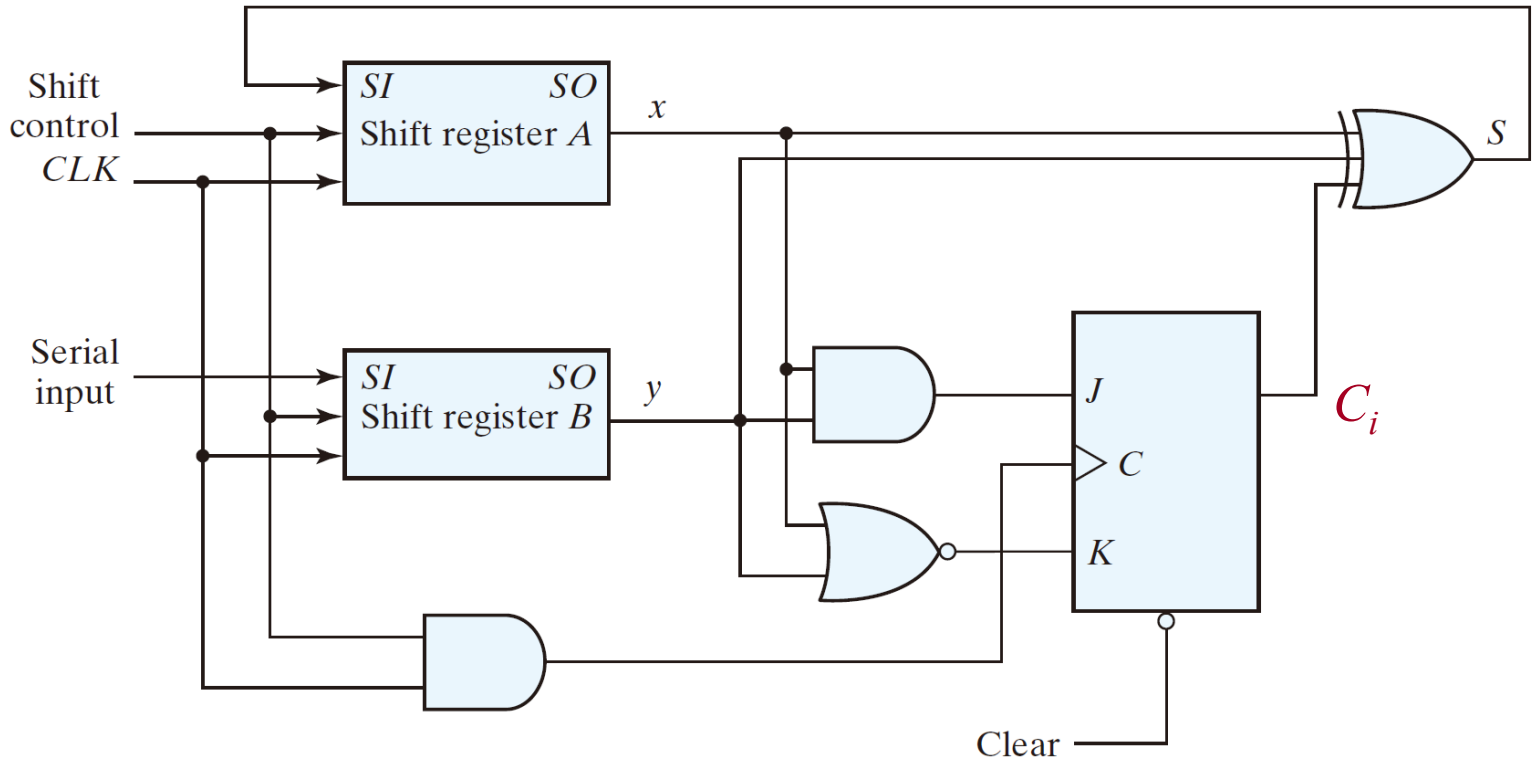


FIGURE 6.6
Second form of serial adder

6.2 Shift Registers (8/12)
Universal Shift Register

- Unidirectional shift register
- Bidirectional shift register
- Universal shift register: 萬用移位暫存器
  - has both direction shifts & parallel load/out capabilities

6.2 Shift Registers (9/12)
Universal Shift Register

- Capability of a universal shift register:
  1. A *clear* control to clear the register to 0.
  2. A *clock* input to synchronize the operations.
  3. A *shift-right* control to enable the shift right operation and the *serial input* and *output* lines associated w/ (with) the shift right.
  4. A *shift-left* control to enable the shift left operation and the *serial input* and *output* lines associated w/ (with) the shift left.
  5. A *parallel-load* control to enable a parallel transfer and the *n parallel input* lines associated w/ the parallel transfer.
  6. *n parallel output* lines.
  7. A control state that leaves the information in the register unchanged in the presence of the clock.

左移、右移
串列/序列 進出
重设 Clear

6.2 Shift Registers (10/12)
Universal Shift Register

- Example: 4-bit universal shift register

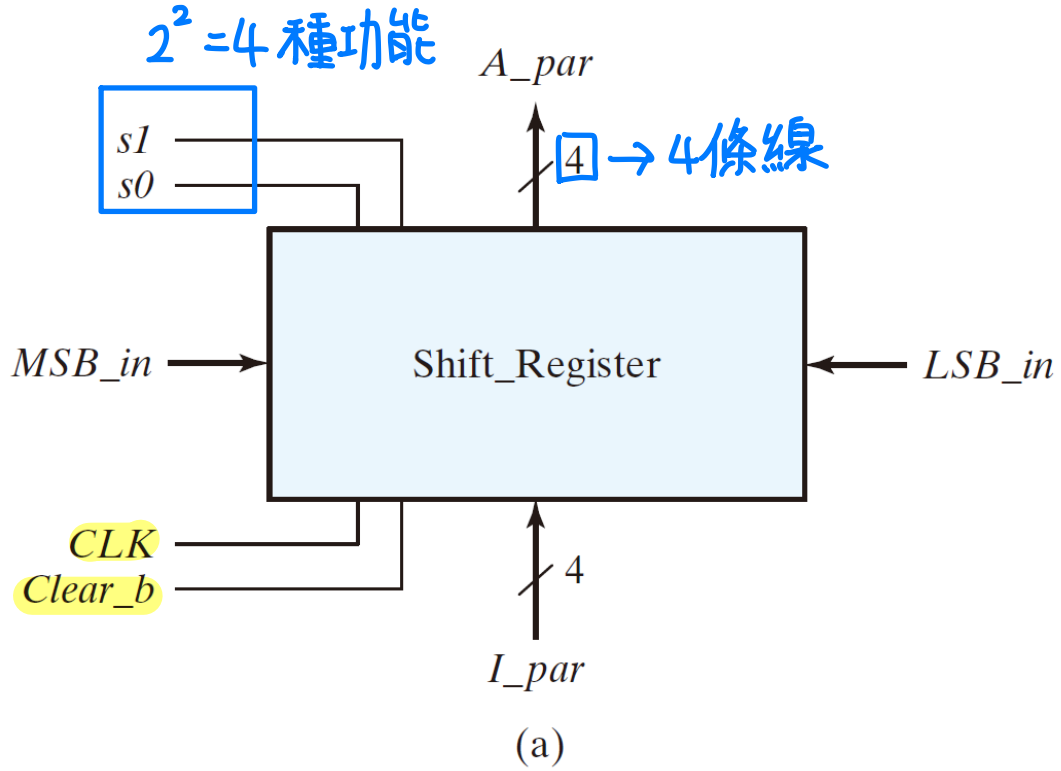


FIGURE 6.7
Four-bit universal shift register

Table 6.3
Function Table for the Register of Fig. 6.7

Mode Control		
s_1	s_0	Register Operation
0	0	No change
0	1	Shift right 右移
1	0	Shift left 左移
1	1	Parallel load 並列輸入

6.2 Shift Registers (11/12)
Universal Shift Register

Mode Control		
s_1	s_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

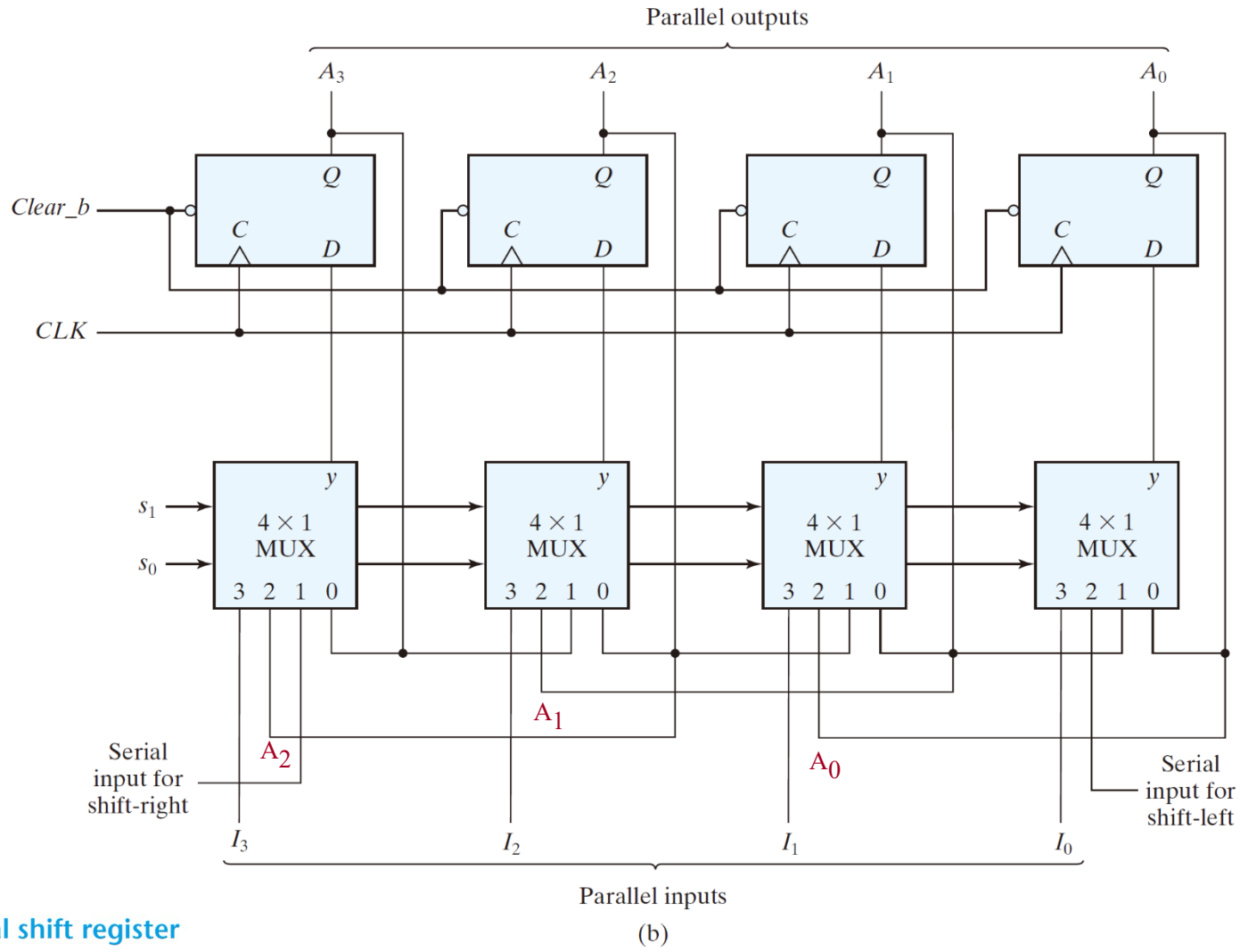
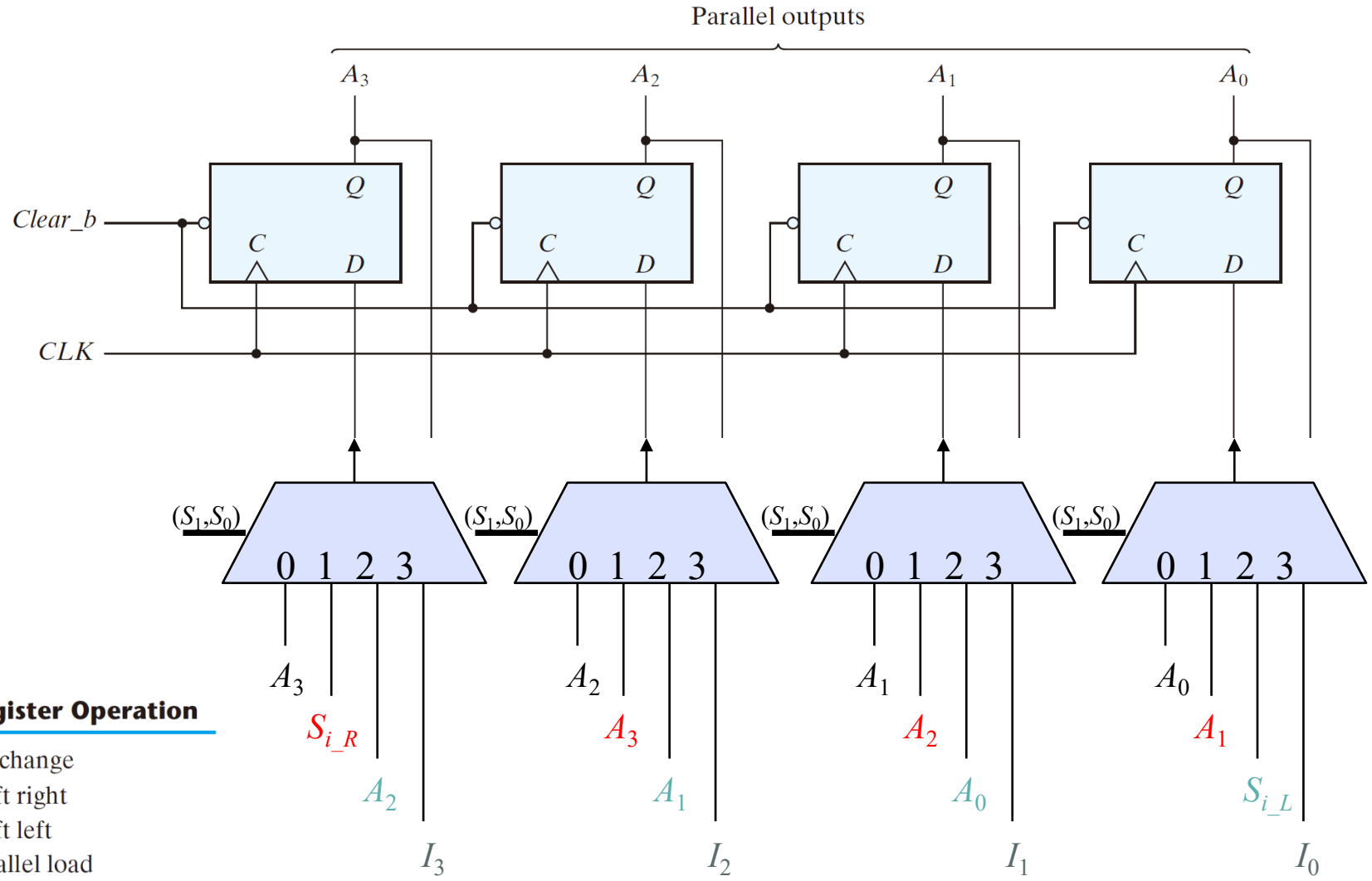


FIGURE 6.7
Four-bit universal shift register

6.2 Shift Registers (12/12)
Universal Shift Register



Mode Control		
s_1	s_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

6.3 Ripple Counters (1/7)

漣波计數器

- Counter
  - A register that goes through a prescribed sequence of states.
  - Upon the application of input pulses, the sequence of states may follow the binary number sequence (→ Binary counter) or any other sequence of states
    - The input pulses may be clock pulses or originate from some external source
- Categories of counters

(非同步计數器)

1. Ripple counters (漣波計數器)

   The flip-flop output transition serves as a source for triggering other flip-flops

   ⇒ no common clock pulse (not synchronous)

2. Synchronous counters (同步計數器)

   The CLK inputs of all flip-flops receive a common clock

6.3 Ripple Counters (2/7)
Binary Ripple Counter

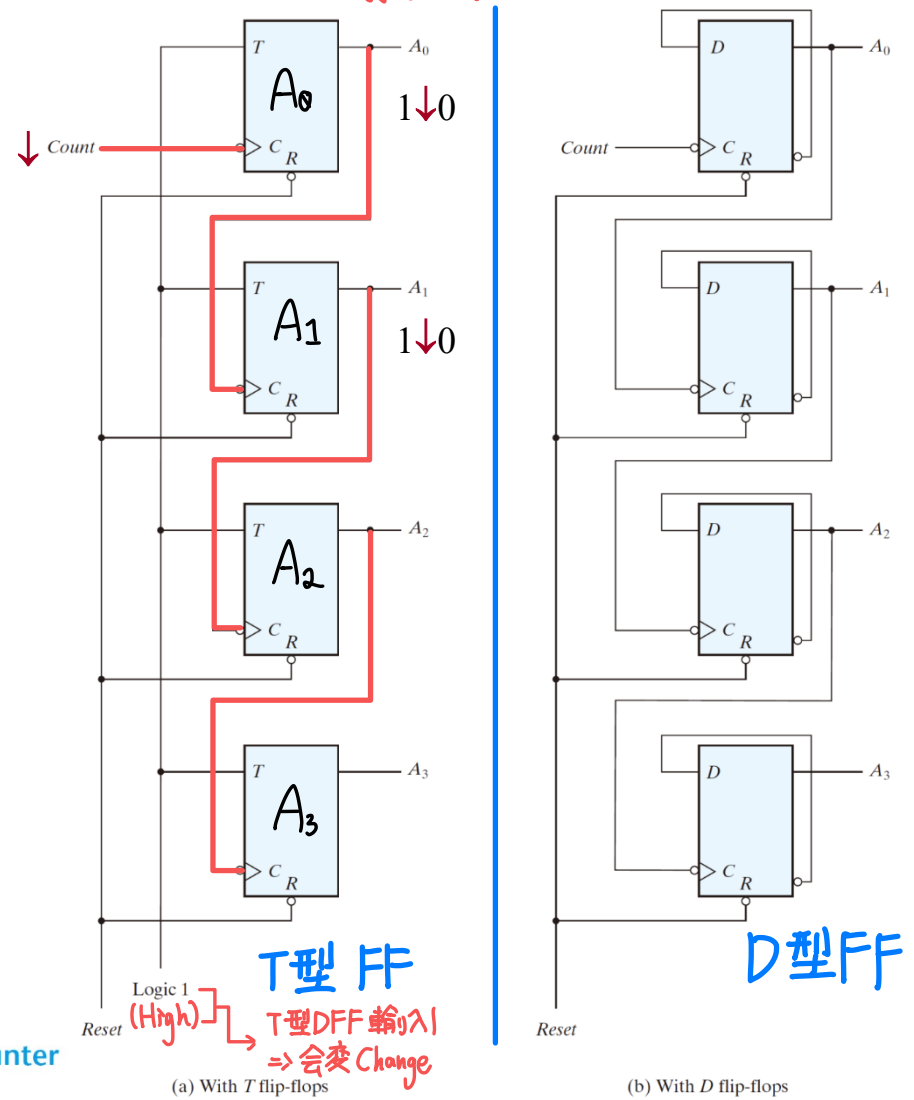
- Example: 4-bit binary ripple counter
  - Binary count sequence: 4-bit

Table 6.4
Binary Count Sequence

A_3	A_2	A_1	A_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0

6.3 Ripple Counters (3/7)
Binary Ripple Counter

功能都一樣!
(負緣觸發)



T型 FF

T型DFF輸入1
=>会变 Change

D型FF

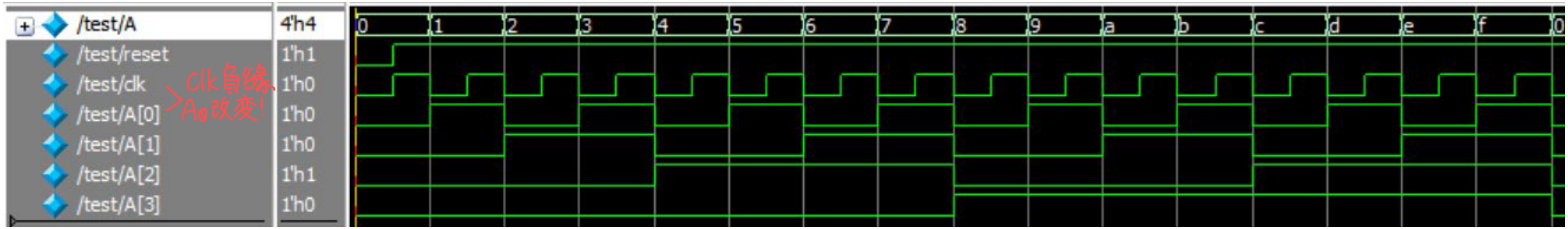
典型的Ripple Counter

(注意Clk! DFF們並沒有共享Clk)
(Not a Synchronous Counter!)

FIGURE 6.8
Four-bit binary ripple counter

6.3 Ripple Counters (4/7)
Binary Ripple Counter

- Timing diagram (with *T* flip-flops)



<key 速度会变慢!>

6.3 Ripple Counters (5/7)
BCD Ripple Counter

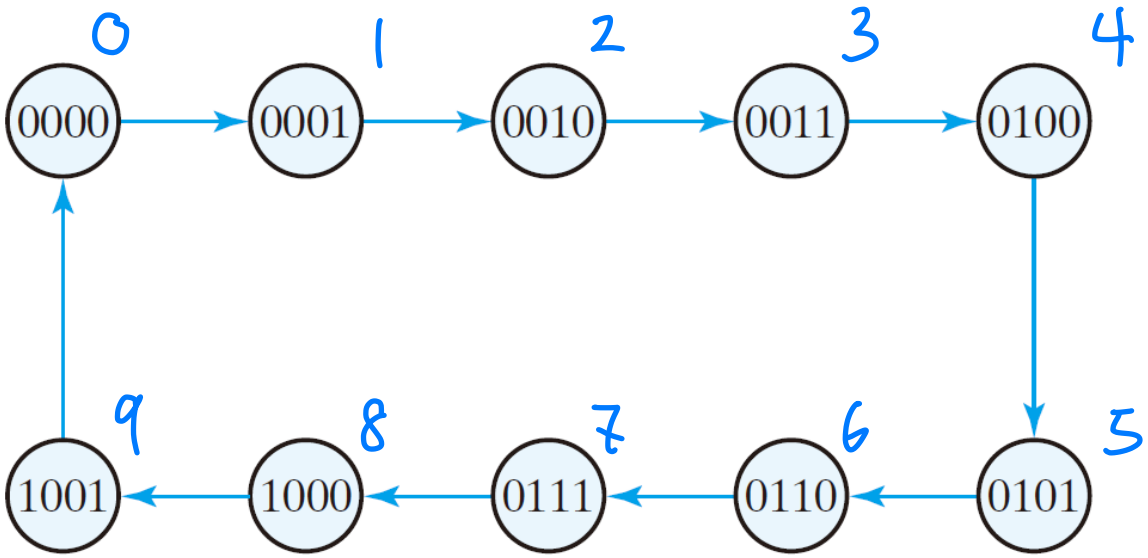


FIGURE 6.9
State diagram of a decimal BCD counter

6.3 Ripple Counters (6/7)
BCD Ripple Counter BCD 漣波计數器 (BCD非同步计數器)

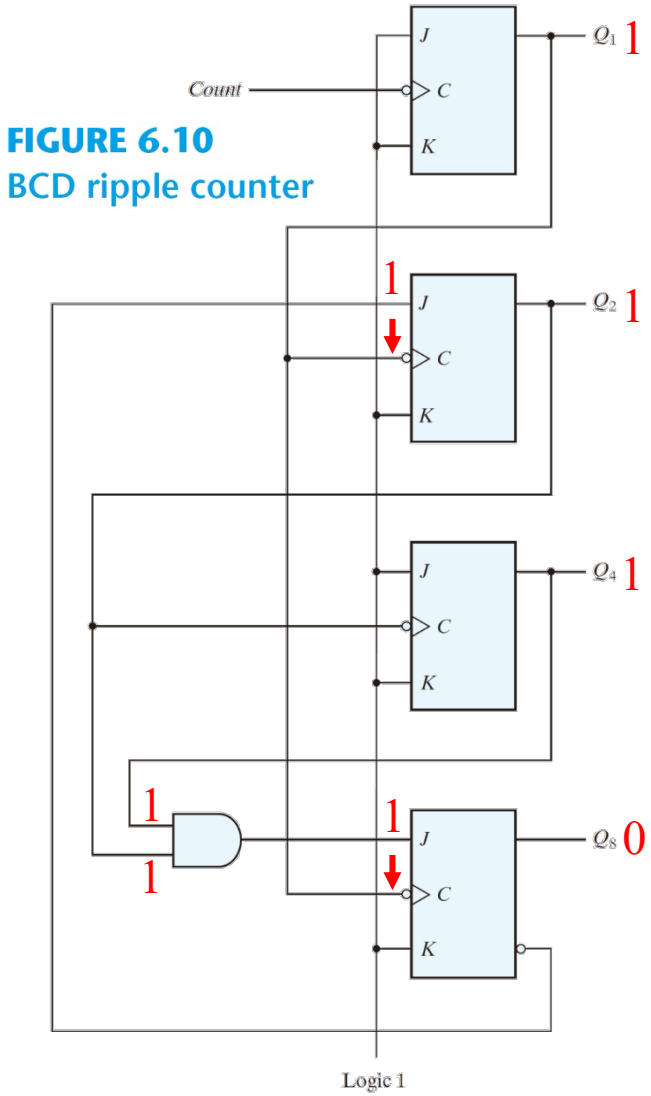
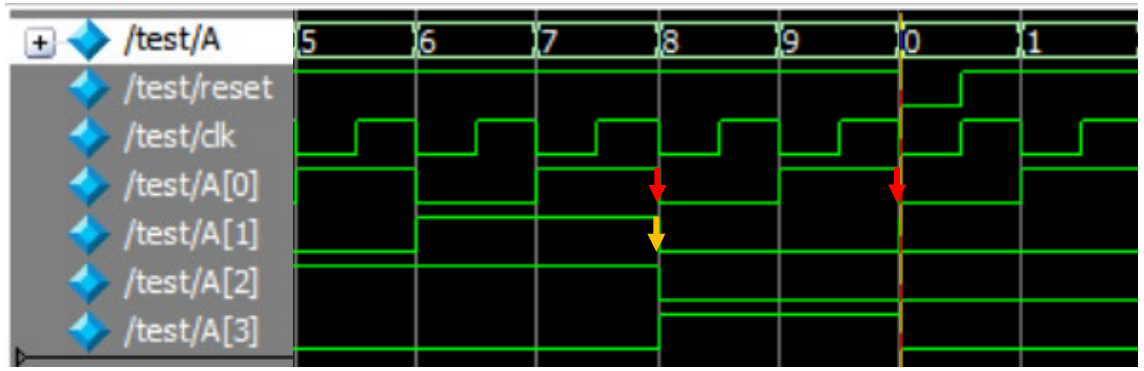


FIGURE 6.10
BCD ripple counter

- Case 1: 0111 (left figure)



- Case 2: 1001 (right figure)

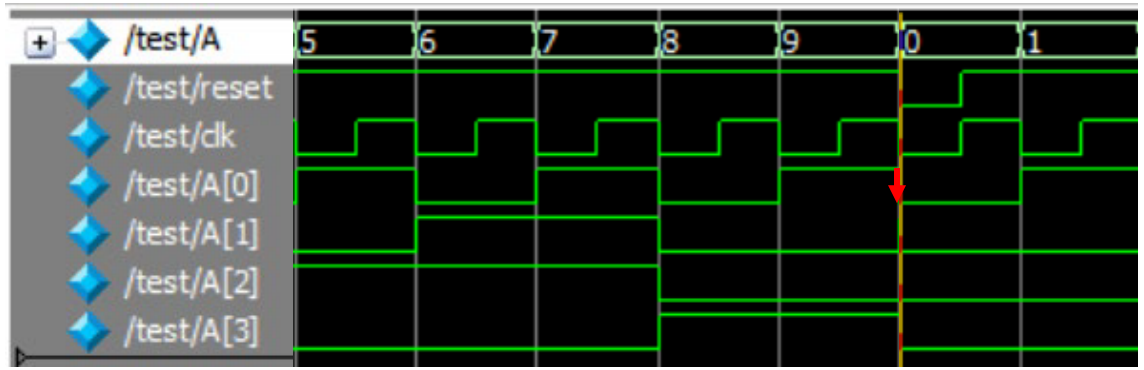
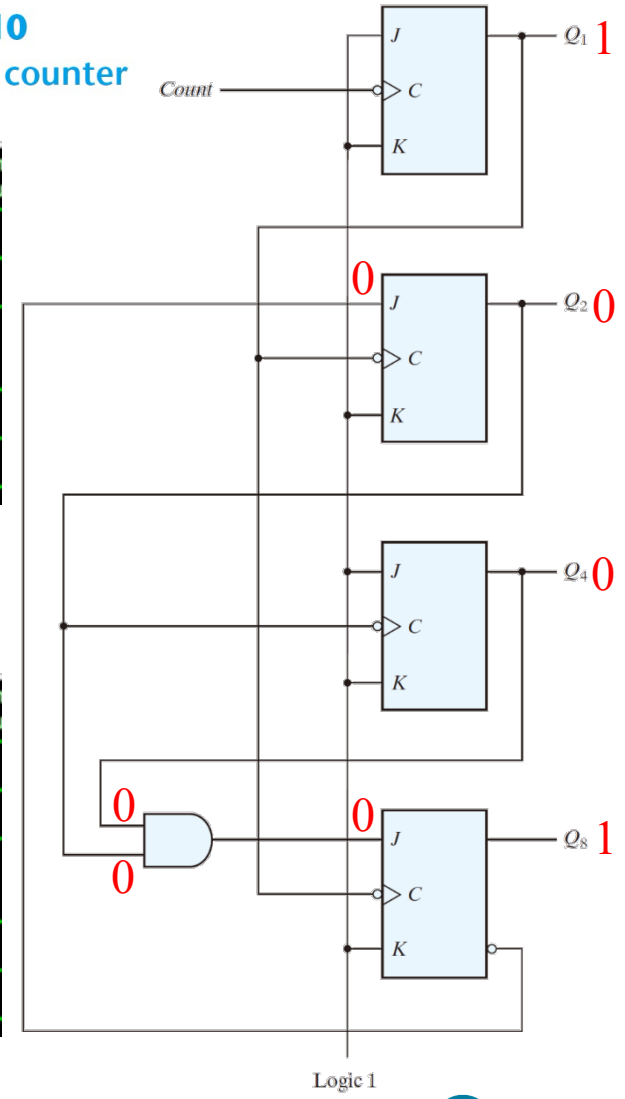


FIGURE 6.10
BCD ripple counter



6.3 Ripple Counters (7/7)
BCD Ripple Counter

- Three-decade BCD counter 一个BCD计数器→数一个十进制

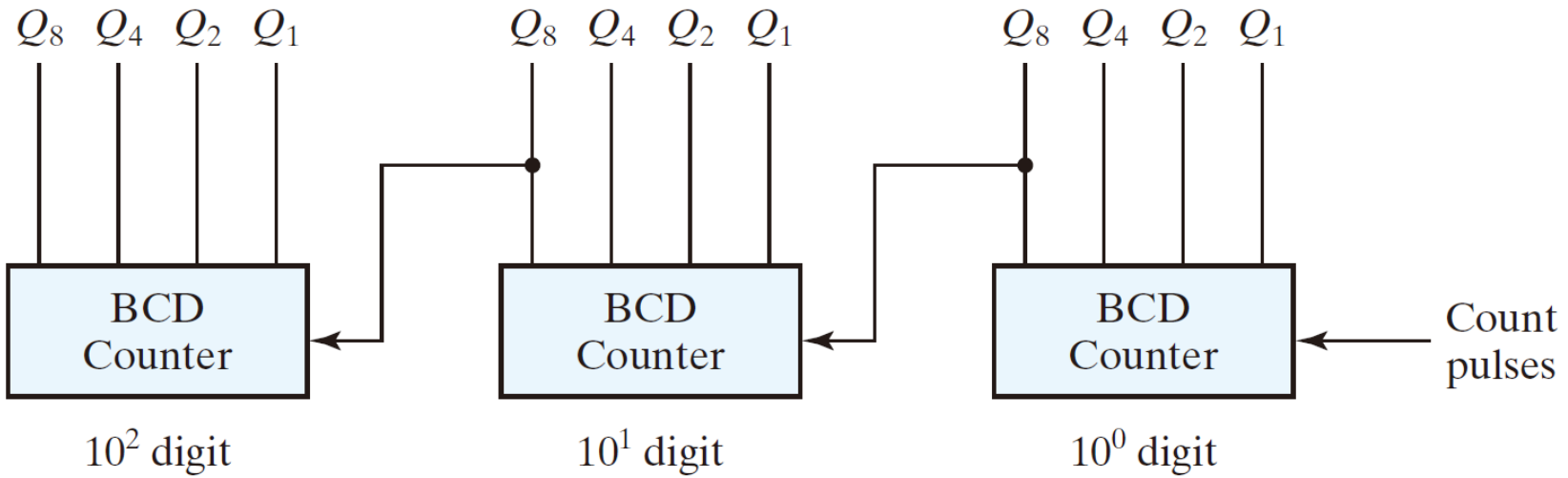


FIGURE 6.11
Block diagram of a three-decade decimal BCD counter

數三个十進制

6.4 Synchronous Counters (1/8)

- Synchronous counter 同步计數器
  - A common clock triggers all flip-flops simultaneously. → 所有正反器共享一个CLK.
- Design procedure
  - Apply the same procedure of synchronous sequential ckts (circuits).
  - Synchronous counter is simpler than general synchronous sequential ckts.

设计同步计數器 將会比 设计序項邏輯來的簡單！

6.4 Synchronous Counters (2/8)
Binary Counter

4個 DFF ⇒ $2^4 = 16$ (0~15)

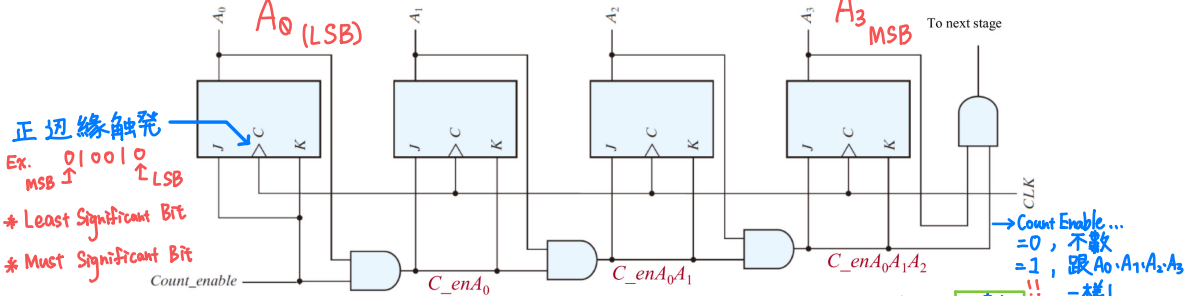
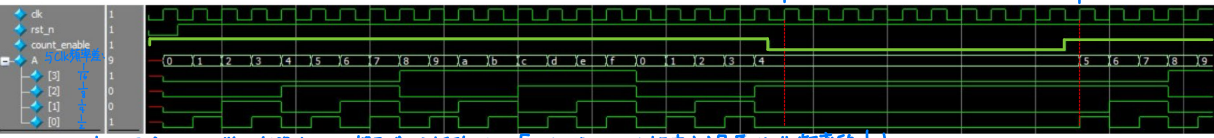


FIGURE 6.12
Four-bit synchronous binary counter

JK正反器的J、K都接一起⇒當成T型正反器！

Count Enable為 0，不數！



↳假設不把他當計數器，每個Bit都看成一個新的Clk ⇒「除頻器」，A_0的頻率就是原始Clk頻率的$\frac{1}{2}$！

6.4 Synchronous Counters (3/8)
Up/Down Binary Counter

上數，每一級的輸入
由前一級的輸出Q決定。
下數，每一級的輸入
由前一級的輸出Q'決定。

只能同時有一個
是1！

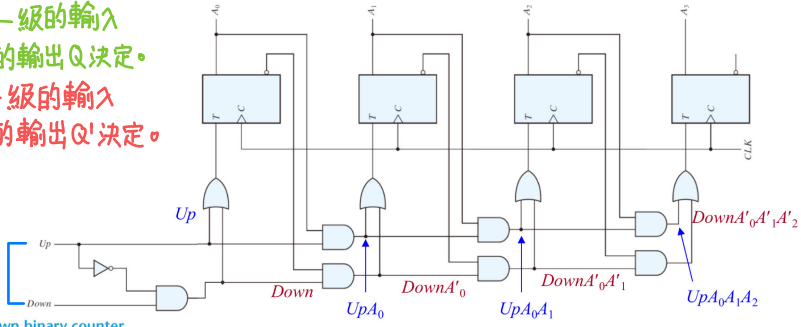
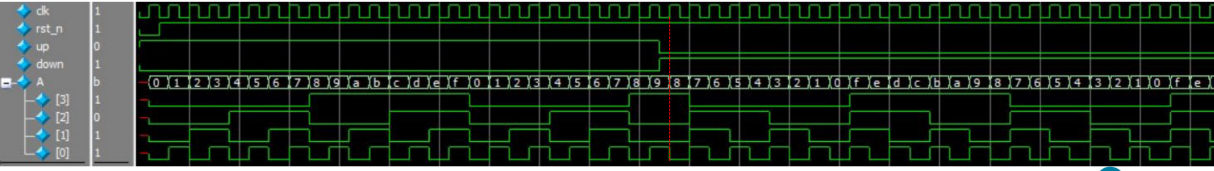


FIGURE 6.13
Four-bit up–down binary counter



6.4 Synchronous Counters (4/8)

BCD Counters → 只有0~9而已!

Table 6.5

State Table for BCD Counter.

Present State					Next State					Output	Flip-Flop Inputs			
Q_8	Q_4	Q_2	Q_1	DEC	Q_8	Q_4	Q_2	Q_1	DEC	y	T_{Q8}	T_{Q4}	T_{Q2}	T_{Q1}
0	0	0	0	0	0	0	0	1	1	0	0	0	0	1
0	0	0	1	1	0	0	1	0	2	0	0	0	1	1
0	0	1	0	2	0	0	1	1	3	0	0	0	0	1
0	0	1	1	3	0	1	0	0	4	0	0	1	1	1
0	1	0	0	4	0	1	0	1	5	0	0	0	0	1
0	1	0	1	5	0	1	1	0	6	0	0	0	1	1
0	1	1	0	6	0	1	1	1	7	0	0	0	0	1
0	1	1	1	7	1	0	0	0	8	0	1	1	1	1
1	0	0	0	8	1	0	0	1	9	0	0	0	0	1
1	0	0	1	9	0	0	0	0	0	1	1	0	0	1

- Simplified functions:

$$T_{Q1} = 1$$
$$T_{Q2} = Q'_8 Q_1$$
$$T_{Q4} = Q_2 Q_1$$
$$T_{Q8} = Q_8 Q_1 + Q_4 Q_2 Q_1$$
$$y = Q_8 Q_1$$

6.4 Synchronous Counters (5/8)
Binary Counter with Parallel Load

Table 6.6
Function Table for the Counter of Fig. 6.14

Clear	CLK	Load	Count	Function
0	X	X	X	Clear to 0
1	↑	1	X	Load inputs
1	↑	0	1	Count next binary state
1	↑	0	0	No change

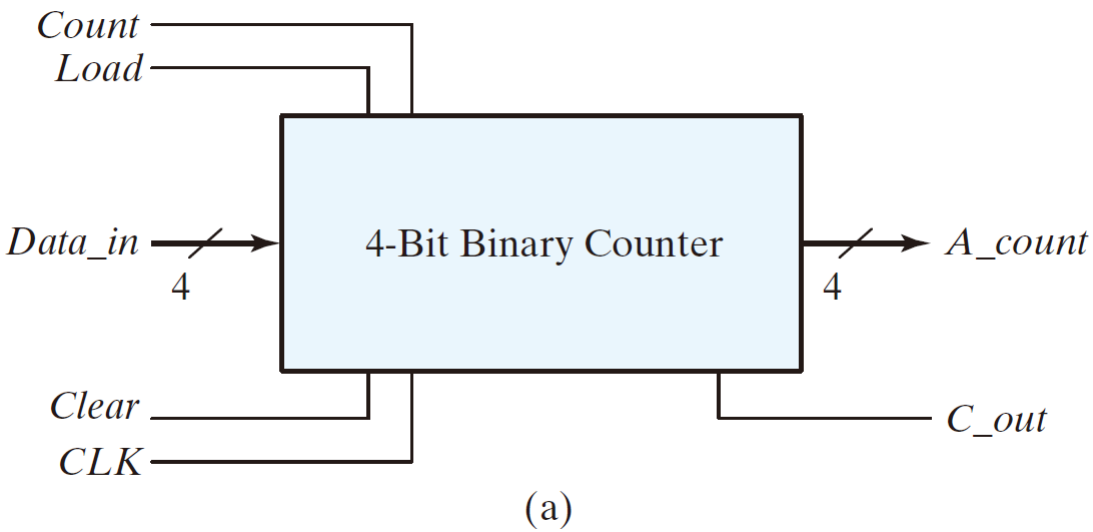


FIGURE 6.14
Four-bit binary counter with parallel load

6.4 Synchronous Counters (6/8)
Binary Counter with Parallel Load

Table 6.6
Function Table for the Counter of Fig. 6.14

Clear	CLK	Load	Count	Function
0	X	X	X	Clear to 0
1	↑	1	X	Load inputs
1	↑	0	1	Count next binary state
1	↑	0	0	No change

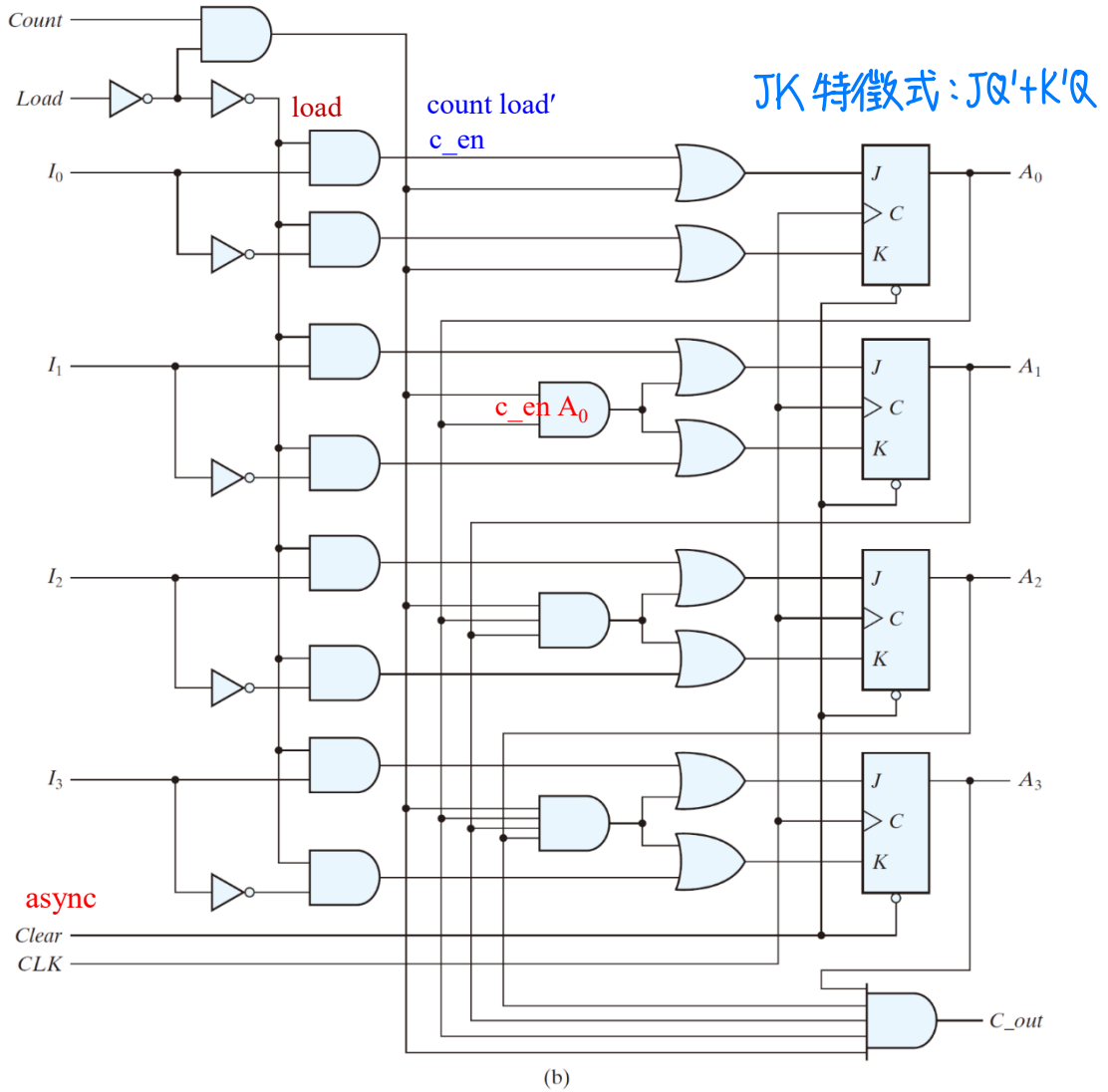


FIGURE 6.14
Four-bit binary counter with parallel loa

6.4 Synchronous Counters (7/8)
Binary Counter with Parallel Load

- Generate any count sequence
  - E.g.: BCD counter ⇐ Counter w/ parallel load

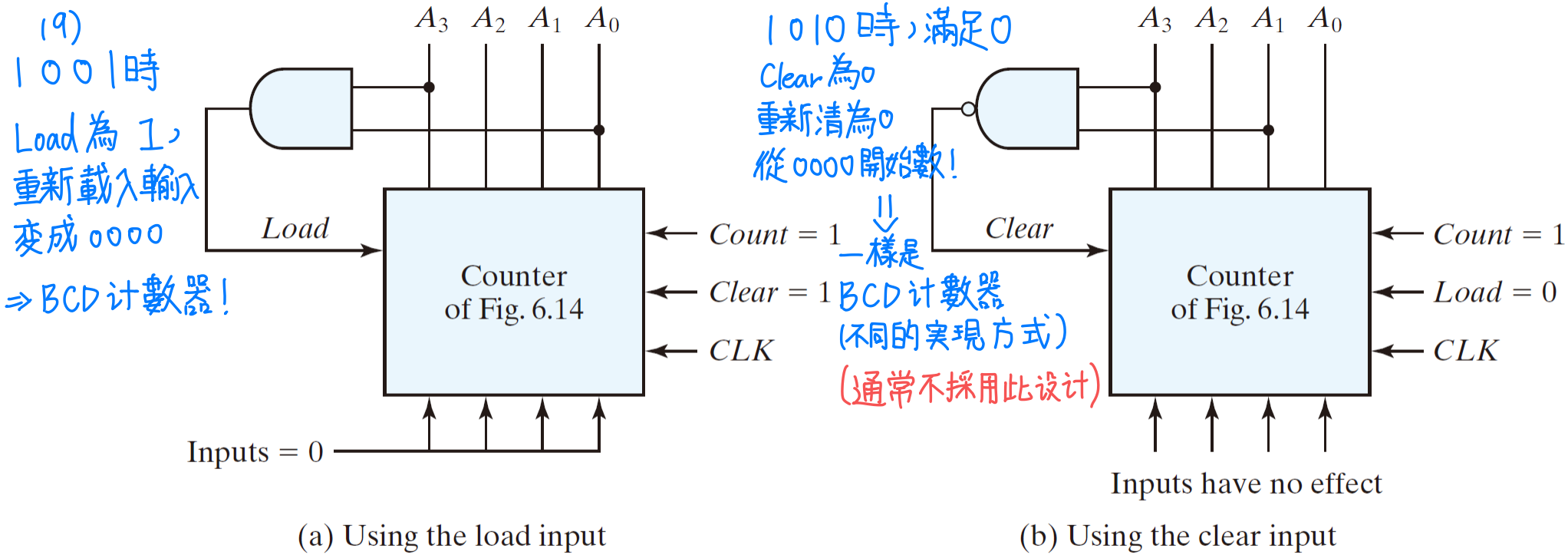


FIGURE 6.15
Two ways to achieve a BCD counter using a counter with parallel load

6.4 Synchronous Counters (8/8)
Binary Counter with Parallel Load

- **Practice Exercise 6.3** How do a ripple counter and a synchronous counter differ in their behavior?
  Answer: All of the flip-flops in a synchronous counter are synchronized by receiving a common clock pulse; only the first stage of a ripple counter receives a clock pulse. The stages of a synchronous counter are updated simultaneous by a common clock; the stages of a ripple counter are updated one at a time. A synchronous counter operates faster than a ripple counter.

6.5 Other Counters (1/12)

- Counters:
  - can be designed to generate any desired sequence of states
- Divide-by-*N* counter (modulo-N counter)
  - a counter that goes through a repeated sequence of *N* states
  - The sequence may follow the binary count or may be any other arbitrary sequence

6.5 Other Counters (2/12)
Counter with Unused States

- n flip-flops $\Rightarrow 2^n$ binary states →增加一个正反器，多一个 2^n 次方的计數
- Unused states
  - states that are not used in specifying the FSM
  - may be treated as don't-care conditions or may be assigned specific next states
- Self-correcting counter
  - Ensure that when a ckt enter one of its unused states, it eventually goes into one of the valid states after one or more clock pulses so it can resume normal operation.

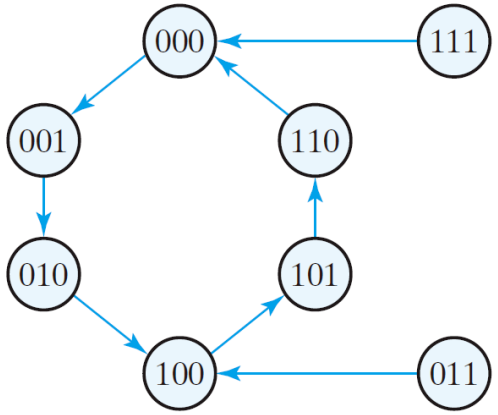
    $\Rightarrow$ Analyze the ckt to determine the next state from an unused state after it is designed

6.5 Other Counters (3/12)
Counter with Unused States

- An example

Table 6.7
State Table for Counter

Present State			Next State			Flip-Flop Inputs					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	1	0	0	1	X	X	1	0	X
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	0	0	0	X	1	X	1	0	X
0	1	1	1	0	0	1	1	1	1	0	1
1	1	1	0	0	0	1	1	1	1	0	1



(b) State transition diagram

Two unused states: 011 & 111

A \ BC	00	01	11	10
0	0	0	X=1	1
1	X	X	X=1	X

$J_A = B$

A \ BC	00	01	11	10
0	X	X	X=1	X
1	0	1	X=1	X

$K_A = B$

A \ BC	00	01	11	10
0	0	1	X=1	X
1	0	1	X=1	X

$J_B = C$

A \ BC	00	01	11	10
0	X	X	X=1	1
1	X	X	X=1	1

$K_B = 1$

A \ BC	00	01	11	10
0	1	X	X=0	0
1	1	X	X=0	0

$J_C = B'$

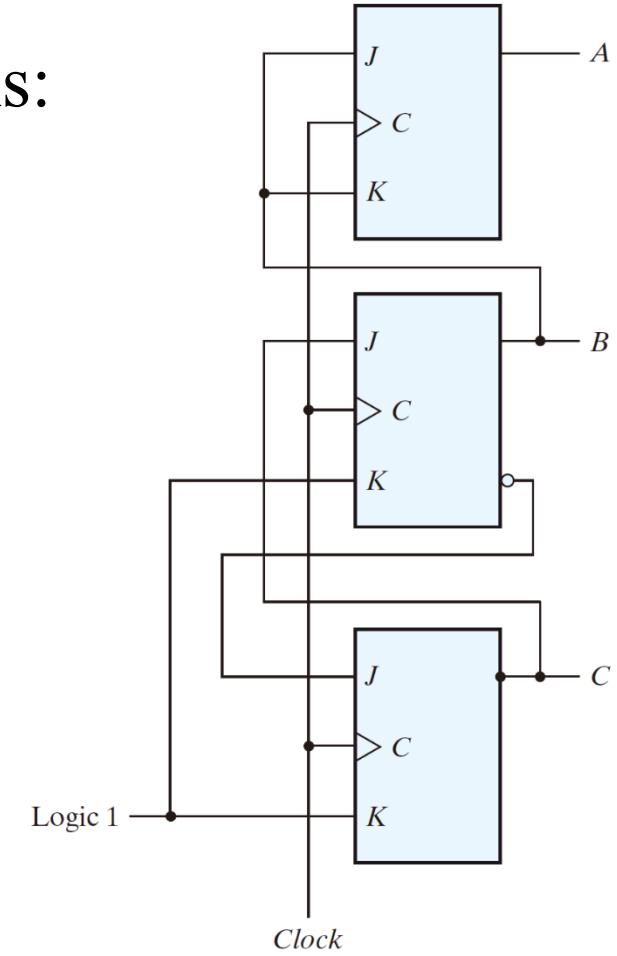
A \ BC	00	01	11	10
0	X	1	X=1	X
1	X	1	X=1	X

$K_C = 1$

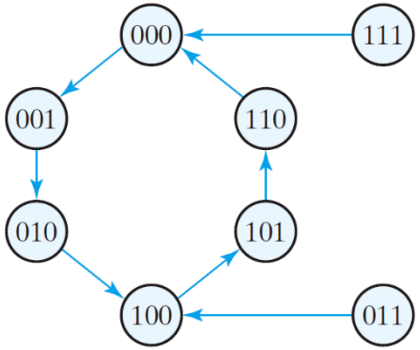
6.5 Other Counters (4/12)
Counter with Unused States

- The logic diagram & state diagram of the circuit
- The simplified flip-flop input equations:

$$J_A = B, \quad K_A = B$$
$$J_B = C, \quad K_B = 1$$
$$J_C = B', \quad K_C = 1$$



(a) Logic circuit diagram



(b) State transition diagram

FIGURE 6.16
Counter with unused states

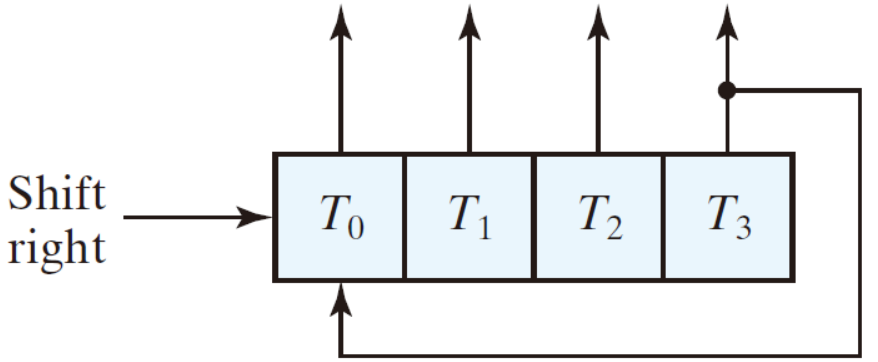
6.5 Other Counters (5/12)
Ring Counter 環形計數器

- A circular shift register with only one flip-flop being set at any particular time, all others are cleared, i.e., initial value = 1 0 0 … 0
- The single bit is shifted from one flip-flop to the next to produce the sequence of timing signals.

6.5 Other Counters (6/12)
Ring Counter

- A 4-bit ring counter

A_2	A_2	A_1	A_0
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

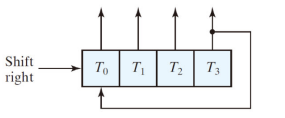


(a) Ring-counter (initial value = 1000)

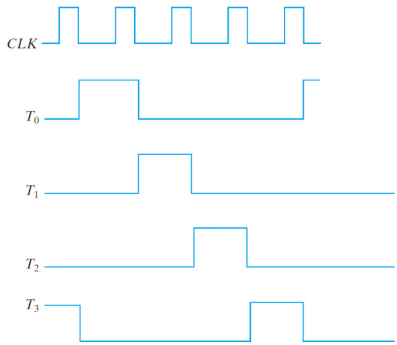
FIGURE 6.17
Generation of timing signals

6.5 Other Counters (7/11)
Ring Counter

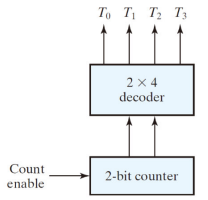
- Approaches for the generation of 2^n timing signals to control the sequence of operations in a digital system
  1. a shift register w/ 2^n flip-flops
  2. an n-bit binary counter together w/ an n-to-2^n-line decoder



(a) Ring-counter (initial value = 1000)



(b) Sequence of four timing signals



(c) Counter and decoder

FIGURE 6.17
Generation of timing signals

6.5 Other Counters (8/11)
Johnson Counter

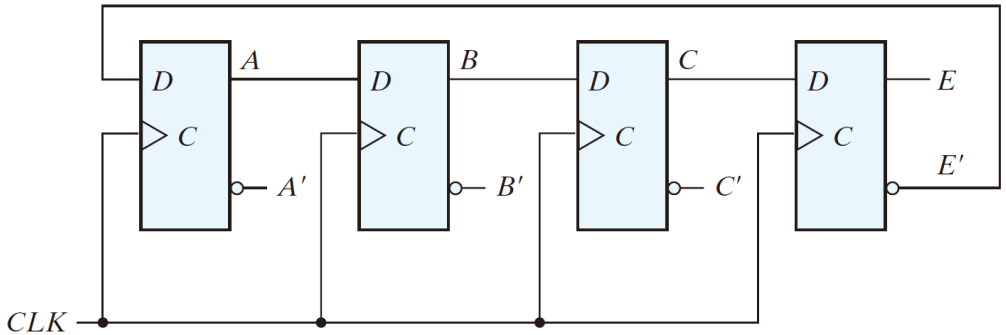
- Ring counter vs. Switch-tail (切換-尾端) ring counter
  - Ring counter
    - A k-bit ring counter circulates a single bit among the flip-flops to provide k distinguishable states.
  - Switch-tail ring counter
    - A circular shift register w/ the complement output of the last flip-flop connected to the input of the first flip-flop
    - A k-bit switch-tail ring counter will go through a sequence of $2k$ distinguishable states. (initial value = 0 0 … 0)

Ring Counter: K个→數K个狀態、初值確保一个為1，其他都是0。

Switch-tail Counter：K个→數2K个狀態、初值確保全部都是0！
(Johnson Counter)

6.5 Other Counters (10/12)
Johnson Counter

- An example: Switch-tail ring counter



(a) Four-stage switch-tail ring counter

Sequence number	Flip-flop outputs A	B	C	E	AND gate required for output
1	0	0	0	0	$A'E'$
2	1	0	0	0	AB'
3	1	1	0	0	BC'
4	1	1	1	0	CE'
5	1	1	1	1	AE
6	0	1	1	1	$A'B$
7	0	0	1	1	$B'C$
8	0	0	0	1	$C'E$

要有8个 AND Gates，
每个 Gates 都是兩輸入，來確保結果！

(b) Count sequence and required decoding

FIGURE 6.18
Construction of a Johnson counter

6.5 Other Counters (11/12)
Johnson Counter

- Johnson counter
  - a k-bit switch-tail ring counter + $2k$ decoding gates
  - provide outputs for $2k$ timing signals
    - E.g.: 4-bit Johnson counter

Sequence number	Flip-flop outputs				AND gate required for output
	A	B	C	E	
1	0	0	0	0	$A'E'$
2	1	0	0	0	AB'
3	1	1	0	0	BC'
4	1	1	1	0	CE'
5	1	1	1	1	AE
6	0	1	1	1	$A'B$
7	0	0	1	1	$B'C$
8	0	0	0	1	$C'E$

(b) Count sequence and required decoding

  - The decoding follows a regular pattern:
    - 2 inputs per decoding gate

6.5 Other Counters (11/11)
Johnson Counter

- Disadvantages of the switch-tail ring counter
  - If it finds itself in an unused state, it will persist to circulate in the invalid states and never find its way to a valid state.
  - One correcting procedure is $D_C = (A + C)B$
- Summary
  - Johnson counters can be constructed for any # of timing sequences:

    # of flip-flops = 1/2 (the # of timing signals)

    # of decoding gates = # of timing signals

    2-input per decoding AND gate

Question! How to design a k-bit Johnson counter that generates (2k-1) timing signals?

6.6 HDL for Registers and Counters (1/12)
Shift Register

- Statement: A_par <= {MSB_in, A_par [3: 1]}
  - specifies a concatenation of serial data input for a right shift operation (*MSB_in*) with bits *A_par[3 : 1] of the output data bus*.

6.6 HDL for Registers and Counters (2/12)
Shift Register

- HDL Example 6.1 (Universal Shift Register-Behavioral Model)

Verilog

```
// Behavioral description of a 4-bit universal shift register
// Fig. 6.7 and Table 6.3
module Shift_Register_4_beh (                    // V2001, 2005 syntax
 output reg  [3: 0]      A_par,                  // Register output
 input       [3: 0]      I_par,                  // Parallel input
 input       s1, s0,                             // Select inputs
             MSB_in, LSB_in,                     // Serial inputs
             CLK, Clear_b                        // Clock and active-low clear
);

always @ (posedge CLK, negedge Clear_b)          // V2001, 2005 syntax
 if (Clear_b == 0) A_par <= 4'b000;
 else
  case ({s1, s0})
   2'b00: A_par <= A_par;                        // No change
   2'b01: A_par <= {MSB_in, A_par[3: 1]};        // Shift right
   2'b10: A_par <= {A_par[2: 0], LSB_in};        // Shift left
   2'b11: A_par <= I_par;                        // Parallel load of input
  endcase
endmodule
```

6.6 HDL for Registers and Counters (3/12)
Shift Register

- Variables of type **reg.** retain their value until they are assigned a new value by an assignment statement. Consider the following alternative **case** statement for the shift register model:

```
case ({s1, s0})
   // 2'b00: A_par <= A_par;                    // No change
   2'b01: A_par <= {MSB_in, A_par [3: 1]};     // Shift right
   2'b10: A_par <= {A_par [2: 0], LSB_in};     // Shift left
   2'b11: A_par <= I_par;                      // Parallel load of input
endcase
```

6.6 HDL for Registers and Counters (4/12)
Shift Register

- HDL Example 6.2 (Universal Shift Register-Structural Model)

```
// Structural description of a 4-bit universal shift register (see Fig. 6.7)
module Shift_Register_4_str (                                  // V2001, 2005
  output [3: 0]    A_par,                                      // Parallel output
  input [3: 0]     I_par,                                      // Parallel input
  input            s1, s0,                                     // Mode select
  input            MSB_in, LSB_in, CLK, Clear_b                // Serial inputs, clock, clear
);

// bus for mode control
  wire   [1:0]   select = {s1, s0};

// Instantiate the four stages
  stage ST0 (A_par[0], A_par[1], LSB_in, I_par[0], A_par[0], select, CLK, Clear_b);
  stage ST1 (A_par[1], A_par[2], A_par[0], I_par[1], A_par[1], select, CLK, Clear_b);
  stage ST2 (A_par[2], A_par[3], A_par[1], I_par[2], A_par[2], select, CLK, Clear_b);
  stage ST3 (A_par[3], MSB_in, A_par[2], I_par[3], A_par[3], select, CLK, Clear_b);
endmodule
```

6.6 HDL for Registers and Counters (5/12)
Shift Register

- HDL Example 6.2 (Universal Shift Register-Structural Model)

```
// One stage of shift register
module stage (i0, i1, i2, i3, Q, select, CLK, Clr_b);
  input        i0,          // circulation bit selection
               i1,          // data from left neighbor or serial input for shift-right
               i2,          // data from right neighbor or serial input for shift-left
               i3;          // data from parallel input
  output       Q;
  input [1: 0] select;      // stage mode control bus
  input        CLK, Clr_b;  // Clock, Clear for flip-flops
  wire         mux_out;

// instantiate mux and flip-flop
  wire Clr = ~Clr_b         // Flip-flop has active-high clear signal
                            // but circuit has active-low clear action
  Mux_4x1      M0           (mux_out, i0, i1, i2, i3, select);
  D_flip_flop  M1           (Q, mux_out, CLK, Clr);
endmodule
```

6.6 HDL for Registers and Counters (6/12)
Shift Register

- HDL Example 6.2 (Universal Shift Register-Structural Model)

```
// 4x1 multiplexer              // behavioral model
module Mux_4x1 (mux_out, i0, i1, i2, i3, select);
  output       mux_out;
  input        i0, i1, i2, i3;
  input [1: 0] select;
  reg          mux_out;
  always @ (select, i0, i1, i2, i3)
    case (select)
      2'b00:     mux_out = i0;
      2'b01:     mux_out = i1;
      2'b10:     mux_out = i2;
      2'b11:     mux_out = i3;
  endcase
endmodule

// Behavioral model of D flip-flop
module D_flip_flop (Q, D, CLK, Clr);
  output  Q;
  input   D, CLK, Clr;
  reg     Q;
  always @ (posedge CLK, posedge Clr)
    if (Clr) Q <= 1'b0; else Q <= D;
endmodule
```

6.6 HDL for Registers and Counters (7/12)
Synchronous Counter

HDL Example 6.3 (Synchronous Counter)

```
// Four-bit binary counter with parallel load (V2001, 2005)
// See Figure 6.14 and Table 6.6
module Binary_Counter_4_Par_Load (
  output reg [3: 0]                 A_count,     // Data output
  output                            C_out,       // Output carry
  input [3: 0]                      Data_in,     // Data input
  input                             Count,       // Active high to count
                                    Load,        // Active high to load
                                    CLK,         // Positive-edge sensitive
                                    Clear_b      // Active low
);
assign C_out = Count && (~Load) && (A_count == 4'b1111);
always @ (posedge CLK, negedge Clear_b)
    if (~Clear_b)                   A_count <= 4'b0000;
    else if (Load)                  A_count <= Data_in;
    else if (Count)                 A_count <= A_count + 1'b1;
    else                            A_count <= A_count;  // redundant statement
endmodule
```

6.6 HDL for Registers and Counters (8/12)
Ripple Counter

HDL Example 6.4 (Ripple Counter)

```
// Ripple counter (See Fig. 6.8(b))
'timescale 1ns / 100 ps
module Ripple_Counter_4bit (A3, A2, A1, A0, Count, Reset);
   output A3, A2, A1, A0;
   input Count, Reset;
// Instantiate complementing flip-flop
   Comp_D_flip_flop F0 (A0, Count, Reset);
   Comp_D_flip_flop F1 (A1, A0, Reset);
   Comp_D_flip_flop F2 (A2, A1, Reset);
   Comp_D_flip_flop F3 (A3, A2, Reset);
endmodule
// Complementing flip-flop with delay
// Input to D flip-flop = Q'
module Comp_D_flip_flop (Q, CLK, Reset);
   output   Q;
   input    CLK, Reset;
   reg      Q;
   always @ (negedge CLK, posedge Reset)
   if (Reset) Q <= 1'b0;
   else Q <= #2 ~Q;              // intra-assignment delay
```

6.6 HDL for Registers and Counters (9/12)
Ripple Counter

```
endmodule
// Stimulus for testing ripple counter
module t_Ripple_Counter_4bit;
   reg      Count;
   reg      Reset;
   wire     A0, A1, A2, A3;
// Instantiate ripple counter
   Ripple_Counter_4bit M0 (A3, A2, A1, A0, Count, Reset);
    always
   #5 Count = ~Count;
    initial
      begin
   Count = 1'b0;
   Reset = 1'b1;
   #4 Reset = 1'b0;
     end

    initial #170 $finish;

endmodule
```

6.6 HDL for Registers and Counters (10/12)
Ripple Counter

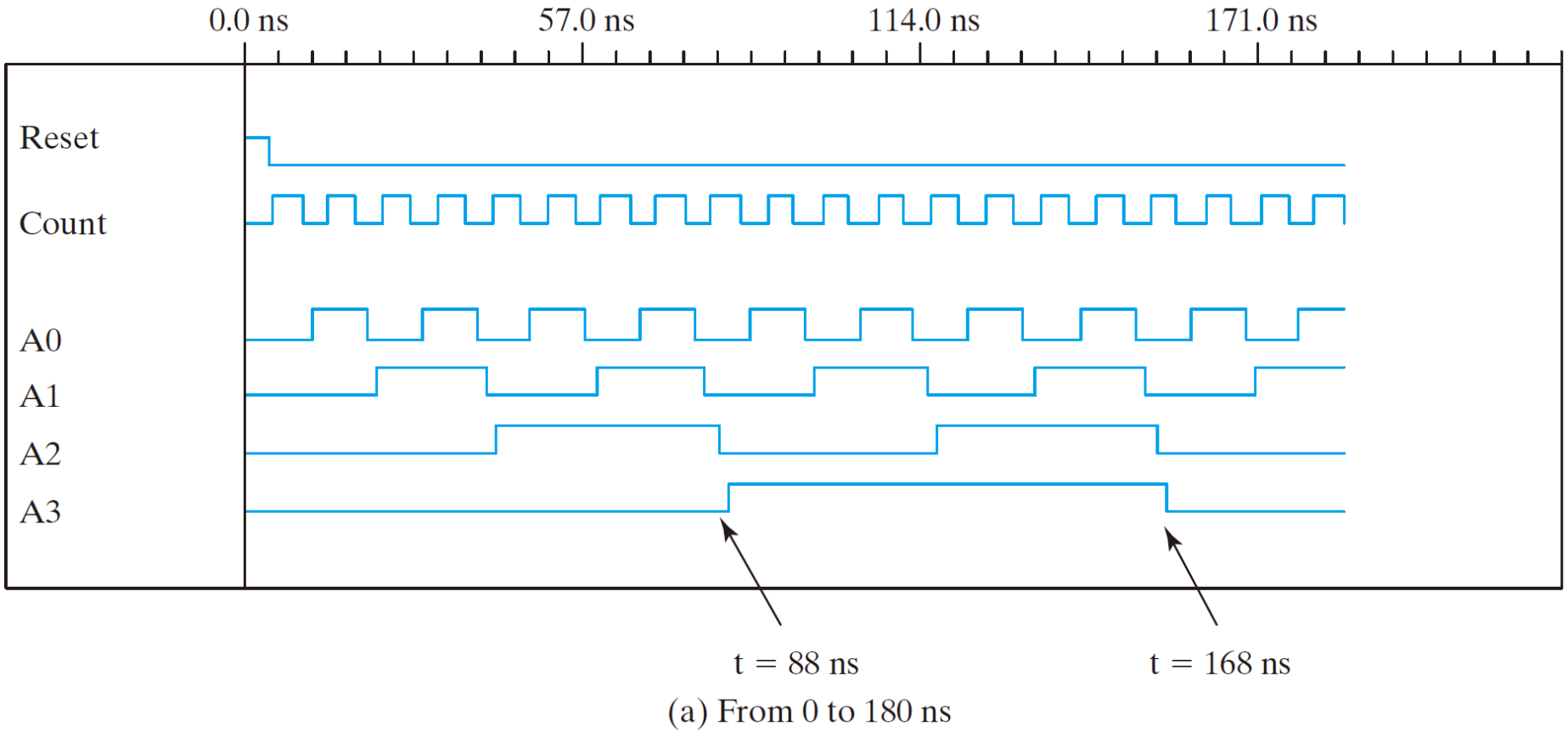
- **Practice Exercise 6.3 – Verilog** The bits of a four-bit ripple counter are labelled A3, A2, A1, and A0. If the flip-flops of the counter are instantiated and interconnected as shown in the text below, the counter fails to function correctly. Find the error in the code

```
Comp_D_flip_flop F0 (A0, Count, Reset);
Comp_D_flip_flop F1 (A1, A0, Reset);
Comp_D_flip_flop F2 (A2, A3, Reset);
Comp_D_flip_flop F3 (A3, A1, Reset);
```

Answer: Flip-flops *F*2 and *F*3 are not clocked correctly. In *F*2, replace *A*3 by *A*1. In *F*3, replace *A*1 by *A*2.

6.6 HDL for Registers and Counters (11/12)
Ripple Counter

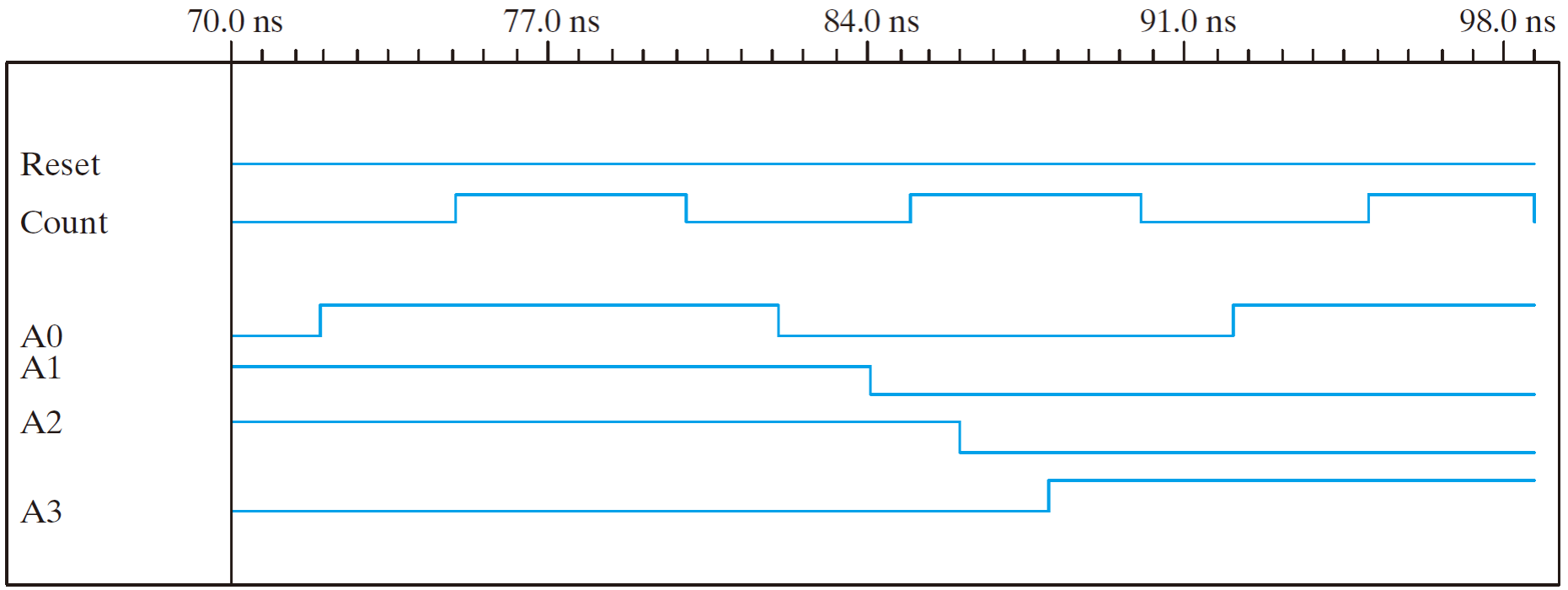
- Simulation Output of HDL Example 6.4



(a) From 0 to 180 ns

6.6 HDL for Registers and Counters (12/12)
Ripple Counter

- Simulation Output of HDL Example 6.4



(b) From 70 to 98 ns

FIGURE 6.19
Simulation output of HDL Example 6.4